ELEC 475: Lab 3 — Pre-Lab

Pet Nose Localization: SnoutNet, AlexNet and VGG16

Tyler Wilson and Michael Greenspan

Oct. 18, 2026

Table of Contents

Introduction 2
Task 1: Environment Isolation (~10 min) 2
Step 1: Create and Activate the `.venv` 3
Step 2: Install the Dependencies 3
Task 2: Dataset & Weights Download (~15 min) 4
Task 3: Custom Dataset (~30 min) 5
Step 1: Implement `PetNoseDataset` 5
Step 2: Reality-Check Your Dataset 6
Task 4: Model Implementation (~30 min) 6
Step 1: Implement SnoutNet 7
Step 2: Implement SnoutNet-A and SnoutNet-V 7
Step 3: Test Your Models 8
Task 5: Verification (~10 min) 8
Step 1: Load the Provided Weights 8
Step 2: Reproduce a Known Prediction 9
Step 3: Pre-Lab Readiness Checklist 9

Introduction

This Pre-Lab will fully prepare you for the Lab 3 in-lab session on Oct. 22nd. In Lab 3 you will evaluate and compare three models that localize pet noses: SnoutNet, SnoutNet-A (AlexNet backbone) and SnoutNet-V (VGG16 backbone), and then combine them into an ensemble.

You will **not** train any models for this Pre-Lab. Trained weights for all three models are provided. Instead, you will write the code that loads the dataset and builds the three models, and confirm that the provided weights load into your models correctly. The mandatory pre-lab deliverables consist of five tasks, completed prior to attending the in-lab session:

- Task 1: Environment Isolation
- Task 2: Dataset & Weights Download
- Task 3: Custom Dataset
- Task 4: Model Implementation
- Task 5: Verification

Your work goes into two files in the starter repository, plus a short check script you write in Task 3:

```
lab3_template/
├── checkpoints/          # Task 2: provided weights go here
├── data/                 # Task 2: dataset goes here
├── models/
│   ├── __init__.py
│   └── snoutnet.py       # TODO (Task 4): the three models
├── utils/
│   ├── __init__.py
│   └── dataset.py        # TODO (Task 3): PetNoseDataset
├── plots/
├── README.md
└── requirements.txt
```

You may use a large language model (e.g., ChatGPT, Claude) for this Pre-Lab. When prompting, be explicit and precise about what you want. Test and fact-check every piece of code the model generates: verify imports, function names, arguments and tensor shapes, and make sure it runs correctly.

Task 1: Environment Isolation (~10 min)

Create a new `.venv` via the Conda base Python interpreter and install the Lab 3 dependencies (`pip install -r requirements.txt`).

Lab 3 uses the same **strict environment isolation** as Lab 2. Create a **new** `.venv` inside your Lab 3 folder rather than reusing your Lab 2 environment, so that packages installed for one lab never break another.

Step 1: Create and Activate the `.venv`

From your Conda `(base)` prompt, navigate to your Lab 3 project root directory:

```
# Navigate to lab folder
cd path/to/lab3_template

# Confirm the parent Python is 3.10 or newer (3.11 recommended)
python --version

# Create the isolated environment
python -m venv .venv

# On macOS / Linux / WSL Bash (Course Standard):
source .venv/bin/activate

# On Windows PowerShell:
.\.venv\Scripts\Activate.ps1
```

[!TIP] **Script Execution Error:** If PowerShell reports that running scripts is disabled on this system, run `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` once, then activate again.

Step 2: Install the Dependencies

Install the dependencies, then confirm that PyTorch is installed. The last command should print your PyTorch version.

```
# Upgrade package installer
python -m pip install --upgrade pip

# Install Lab 3 dependencies
pip install -r requirements.txt

# Confirm PyTorch is installed
python -c "import torch; print(torch.__version__)"
```

[!TIP] **Mixed-Version Errors:** If imports fail with errors such as `cannot import name '_imaging' from 'PIL'` or `No module named 'numpy.core._multiarray_umath'`, your `.venv` contains packages built for a different Python version. Close every terminal and editor using the environment, delete the `.venv` folder completely, and repeat Step 1.

Task 2: Dataset & Weights Download (~15 min)

Download the oxford-iiit-pet-noses dataset into `./data/` and the three trained weight files into `./checkpoints/`. Both are needed for Task 5 and for the in-lab session.

Start both downloads first, since the files are large.

The oxford-iiit-pet-noses Dataset

- **Origin:** ELEC 475 re-annotation of the Oxford-IIIT Pet Dataset (Parkhi, Vedaldi, Zisserman & Jawahar, University of Oxford Visual Geometry Group, 2012).
- **Original Dataset Website:** robots.ox.ac.uk/~vgg/data/pets
- **Course Download:** ELEC 475 Google Drive (https://drive.google.com/drive/folders/1-IoMxHkbaWlW6w3YA-Fn9ZDv_T6L3pZ4?usp=sharing). Download only; do not modify anything within the shared Drive.
- **Description:** 7,390 images of 37 cat and dog breeds. 6,192 of them are labelled with the pixel location of the pet's nose: 5,494 for training and 698 for testing.
- **Why Selected for Lab 3:** Nose localization is a **regression** task: the model predicts a continuous (u, v) position rather than a class label.

Extract the dataset so that your folder matches:

```
data/oxford-iiit-pet-noses/
├── images-original/
│   └── images/          # 7,390 .jpg images
├── train_noses.txt      # 5,494 training labels
└── test_noses.txt       # 698 test labels
```

Note the underscores in the label file names. Each line holds an image file name and the nose position in that image:

```
beagle_145.jpg,"(198, 304)"
shiba_inu_136.jpg,"(182, 203)"
english_cocker_spaniel_181.jpg,"(145, 293)"
```

The ordered pair is (u, v) in pixels of the **original** image: u is the column (x) and v is the row (y). Images vary in size, and a few are grayscale or RGBA rather than RGB.

The Trained Model Weights

Download the three weight files from the course Drive (https://drive.google.com/drive/folders/1-OKNBTc_ptVeb49u-2QfNxbb8n2sqKGR?usp=sharing) into `./checkpoints/`:

File	Model	Size
`snoutnet.pth`	SnoutNet	~23 MB
`alexnet.pth`	SnoutNet-A (AlexNet backbone)	~230 MB
`vgg16.pth`	SnoutNet-V (VGG16 backbone)	~540 MB

Each file stores a model's trained parameters as a PyTorch `state_dict`: a dictionary that maps each layer's name to its weights. It does **not** contain the model code. To load the weights, your model must have exactly the same layer names and shapes as the model that was trained.

Task 3: Custom Dataset (~30 min)

Implement the `PetNoseDataset` class in `utils/dataset.py` to load each image and its nose label at 227 × 227 resolution, then verify it visually.

Incorrect data loading is the most common source of silent errors in this lab: a bug here produces a model that runs without errors but predicts the wrong locations.

Step 1: Implement `PetNoseDataset`

Your class must follow this interface exactly, since the in-lab code depends on it:

```
dataset = PetNoseDataset(root="data/oxford-iiit-pet-noses",
                         split="train")   # or split="test"
image, label, orig_size = dataset[i]
```

Output	Shape	Contents
`image`	(3, 227, 227)	The image resized to 227 × 227, ImageNet-normalized
`label`	(2,)	The nose position (u, v) in the 227 × 227 image, in pixels
`orig_size`	(2,)	The original image's width and height (W, H)

Implement the class in four parts:

1. **Parse the Label Files:** Split each line into the file name and the (u, v) pair, then strip the quotes and brackets before converting to `float`.
2. **Load Each Image as RGB:** A few images are grayscale, palette or RGBA. Use `Image.open(path).convert("RGB")` so that every image has three channels.
3. **Resize and Rescale the Label:** Resize with PIL using `img.resize((227, 227), Image.BILINEAR)`. Other resize functions (torchvision, OpenCV) produce slightly different pixels, and the provided weights were trained with this one. Resizing to 227 × 227 stretches the width and height by different factors, so u and v must be scaled separately. Note that PIL's `img.size` returns (W, H), not (H, W):

$$u_{227} = u \cdot \frac{227}{W} \qquad v_{227} = v \cdot \frac{227}{H}$$

4. **Convert and Normalize:** Apply `ToTensor()`, then `Normalize` with the ImageNet statistics `MEAN = [0.485, 0.456, 0.406]` and `STD = [0.229, 0.224, 0.225]`. The pretrained AlexNet and VGG16 backbones expect this normalization, and the provided SnoutNet weights were trained with it too.

Return `orig_size` so that predictions can be mapped back to the original image. Lab 3 reports all localization errors in original-image pixels.

Step 2: Reality-Check Your Dataset

Write a short script, `utils/check_dataset.py`, that:

1. Iterates through every sample of both splits (a `DataLoader` with `batch_size=64` is fastest) and confirms that every image has shape (3, 227, 227) and every label lies inside the 227 × 227 frame.
2. Plots eight training images with their labels drawn as dots, and saves the figure to `plots/dataset_check.png`. Undo the normalization before display: multiply by `STD`, then add `MEAN`.

Run it from the lab folder:

```
python -m utils.check_dataset
```

Open `plots/dataset_check.png` and confirm that every dot sits on a nose. If the dots are consistently misplaced, such as mirrored or stretched, check your u/v order and your rescaling.

[!TIP] **Windows Users:** When a `DataLoader` uses `num_workers > 0`, your script's top-level code must sit under `if __name__ == "__main__":`, or Windows will repeatedly re-launch it.

Task 4: Model Implementation (~30 min)

Implement `SnoutNet`, `SnoutNetAlexNet` and `SnoutNetVGG16` in `models/snoutnet.py`. Each takes an image batch of shape (B, 3, 227, 227) and returns a (B, 2) tensor of predicted nose positions.

Use the layer and attribute names below **exactly**. The provided weight files store each parameter under its layer name (for example, `conv1.weight`), so a model with any other names cannot load them.

Model Outputs Are Normalized: All three models predict (u/227, v/227), so their outputs lie roughly between 0 and 1. Multiply by 227 to get pixels in the 227 × 227 image, or by the original (W, H) to get original-image pixels.

Step 1: Implement SnoutNet

SnoutNet has three convolution blocks followed by three fully connected layers. Each convolution block applies a 3 × 3 convolution with `padding=1`, a ReLU, then 4 × 4 max pooling with stride 4.

Layer	Definition	Output Shape
Input		(3, 227, 227)
`conv1` → ReLU → `pool`	`nn.Conv2d(3, 64, 3, padding=1)`	(64, 57, 57)
`conv2` → ReLU → `pool`	`nn.Conv2d(64, 128, 3, padding=1)`	(128, 15, 15)
`conv3` → ReLU → `pool`	`nn.Conv2d(128, 256, 3, padding=1)`	(256, 4, 4)
Flatten		4,096
`fc1` → ReLU	`nn.Linear(4096, 1024)`	1,024
`fc2` → ReLU	`nn.Linear(1024, 1024)`	1,024
`fc3`	`nn.Linear(1024, 2)`	2

All three blocks share a single pooling layer:

```
self.pool = nn.MaxPool2d(kernel_size=4, stride=4, ceil_mode=True)
```

Spatial Dimension Math

With `ceil_mode=True`, the pooling output size is rounded **up**:

$$N_o = \left\lceil \frac{N_i - K}{S} \right\rceil + 1$$

Work through the three pooling steps by hand and confirm that they produce 57, 15 and 4. With PyTorch's default rounding down (`ceil_mode=False`), the first pool produces 56 instead of 57, and the flattened feature vector no longer matches `fc1`. Be prepared to explain this derivation during the in-lab checkoff. SnoutNet has **5,617,794** trainable parameters in total.

Step 2: Implement SnoutNet-A and SnoutNet-V

These two models reuse ImageNet-pretrained backbones from `torchvision`, replacing the final 1000-class layer with a 2-output layer that predicts (u, v):

Class	Backbone
`SnoutNetAlexNet`	`torchvision.models.alexnet`
`SnoutNetVGG16`	`torchvision.models.vgg16`

In both models, replace `classifier[6]` with `nn.Linear(4096, 2)`.

Both constructors take a `pretrained` argument that defaults to `True`. Store the torchvision model as `self.backbone`, and return `self.backbone(x)` from `forward`. When `pretrained=True`, load the ImageNet weights (`AlexNet_Weights.DEFAULT` or

`VGG16_Weights.DEFAULT`). When `pretrained=False`, build the model without them. Use `pretrained=False` whenever you load the provided weights: they replace the ImageNet weights anyway, and it avoids an unnecessary download. Both backbones accept 227 × 227 input without modification.

Step 3: Test Your Models

Add an `if __name__ == "__main__":` block to `models/snoutnet.py` that passes a random tensor of shape (1, 3, 227, 227) through each model and prints its output shape and parameter count. Use `torchinfo.summary` to print SnoutNet's layer-by-layer output shapes, and compare them with the table in Step 1. Then run:

```
python -m models.snoutnet
```

After the SnoutNet summary, you should see:

```
SnoutNet          output (1, 2)  params 5,617,794
SnoutNetAlexNet   output (1, 2)  params 57,012,034
SnoutNetVGG16     output (1, 2)  params 134,268,738
```

[!TIP] **Harmless Warning:** The first line of output may be a `RuntimeWarning` saying that `models.snoutnet` was found in `sys.modules`. It is caused by running a module inside a package with python -m, and can be ignored.

Task 5: Verification (~10 min)

Load the three provided weight files into your models, then confirm that your dataset and SnoutNet together reproduce a known prediction.

Step 1: Load the Provided Weights

Add the following to the end of the `__main__` block in `models/snoutnet.py`, then run `python -m models.snoutnet` again:

```
checks = [(SnoutNet(), "snoutnet.pth"),
          (SnoutNetAlexNet(pretrained=False), "alexnet.pth"),
          (SnoutNetVGG16(pretrained=False), "vgg16.pth")]
for model, name in checks:
    state = torch.load(f"checkpoints/{name}", map_location="cpu",
                       weights_only=True)
    model.load_state_dict(state, strict=True)
    print(f"Loaded {name}")
```

Setting `strict=True` makes loading fail if any layer name or tensor shape differs from the trained model, and the error message lists the mismatched layers. All three files must print `Loaded`.

Step 2: Reproduce a Known Prediction

A model that loads correctly can still give wrong answers if the dataset is wrong. Check the full pipeline on a single test image: load SnoutNet with the provided weights, take sample 0 from `PetNoseDataset(split="test")`, and print the ground truth and the prediction in original-image pixels:

```
from utils.dataset import PetNoseDataset
model = checks[0][0]# SnoutNet (the loop leaves VGG16 in model)
dataset = PetNoseDataset(split="test")
model.eval()
image, label, orig_size = dataset[0]
with torch.no_grad():
    # Model output is normalized: (u/227, v/227)
    pred = model(image.unsqueeze(0))[0]
print("Ground truth:", label / 227 * orig_size)
print("Prediction:  ", pred * orig_size)
```

The ground truth should match the first line of `test_noses.txt`, and the prediction should be within 1 pixel of the expected value below:

```
Ground truth: tensor([311., 152.])
Prediction:   tensor([350.3443, 181.7347])
```

If the ground truth is wrong, check your label parsing and rescaling. If the ground truth is right but the prediction is not, check your resizing and normalization.

Step 3: Pre-Lab Readiness Checklist

Before attending the in-lab session, confirm that:

- Your `.venv` is active, and the PyTorch version check from Task 1 runs without errors.
- The dataset is in `./data/` and all three weight files are in `./checkpoints/`.
- Every dot in `plots/dataset_check.png` sits on a nose.
- `python -m models.snoutnet` prints the expected output shapes and parameter counts, and loads all three weight files.
- Your SnoutNet reproduces the known prediction for test sample 0.

When every item is confirmed, your environment is certified ready for the in-lab session. Bring your working code: everything you do in the lab builds on it.